

TaskFlow – Documentation technique

Architecture & Choix Techniques

1.1 Architecture générale (Full-Stack découplée)

Le projet est construit sur une architecture **découplée** en trois blocs :

Frontend (Next.js) → API (NestJS REST) → Base de données (PostgreSQL sur Neon)

Pourquoi une architecture découplée ?

Objectif : clarté, évolutivité, et maintenance.

- **Séparation des responsabilités :**
 - Le Front gère l'UI/UX et l'expérience utilisateur.
 - L'API gère la logique métier, la sécurité, la validation, et les règles d'accès.
 - La DB gère la persistance et la cohérence des données.
- **Scalabilité :** chaque partie peut évoluer indépendamment (ex : remplacer la DB, ajouter un mobile, scaler l'API sans toucher au Front).
- **Interopérabilité :** une API REST permet à d'autres clients (mobile, outils internes, intégrations) d'utiliser les mêmes endpoints.

Benchmark vs architecture “monolithique” (ex : Next.js fullstack + API routes + DB)

Alternative : tout mettre dans Next.js (API routes + DB + UI).

- **Avantage :** simple au début, moins de projets à gérer.
- **Limites :**
 - logique métier mélangée avec la partie web
 - sécurité/guards moins structurés

- difficile de faire évoluer vers une vraie API entreprise
- jobs/cron + architecture modulaire plus complexe

Choix final : découpler car le projet ressemble plus à un produit “entreprise” (rôles, audit, permissions, modules, cron jobs) qu’à un simple prototype.

1.2 Frontend : Next.js 16 + App Router + Tailwind

Pourquoi Next.js ?

Next.js est choisi car il combine :

- routage moderne (App Router),
- performances (SSR/SSG si besoin),
- DX excellente (TypeScript, bundling, conventions),
- facilité de déploiement (Vercel ou autre).

Benchmark vs React “pur” (Vite + React Router)

React (Vite) :

- plus léger, très rapide à démarrer
- tu dois ajouter toi-même : routing, conventions SSR (si besoin), structuration...

Next.js :

- standard moderne pour apps web structurées
- très bon pour pages dynamiques type dashboard
- permet d’évoluer vers SSR/SEO si nécessaire
- un peu plus “framework” (mais bénéfique à moyen terme)

Choix final : Next.js car l’app est orientée “produit” (dashboard, pages protégées, navigation), et Next facilite la structure et l’évolution.

App Router et Client Components

- **App Router** : architecture plus moderne et claire (routes basées sur dossiers).
- **Client Components** : utilisés là où on a besoin d'interactivité (modals, tables, drawers, actions).
- Les pages protégées utilisent `requireAuth()` pour s'assurer qu'un token est présent côté client.

TailwindCSS

- rapidité de développement UI
- design cohérent et maintenable (classes utilitaires)
- facile à itérer (important en période de délai)

Communication API via fetch (wrapper apiFetch)

- Centralisation des erreurs (status, messages)
- Réduction du code répétitif
- Facilite l'ajout futur d'un refresh token automatique

1.3 Backend : NestJS + architecture modulaire + JWT + Guards + DTO

Pourquoi NestJS ?

NestJS est choisi car il est très adapté aux APIs "entreprise" :

- **Architecture modulaire** (Modules / Controllers / Services)
- **Injection de dépendances** (maintenabilité + testabilité)
- **Guards** (sécurité et règles d'accès propres)
- **Pipes** (validation DTO)
- **Support natif** de JWT, Passport, Cron jobs

Benchmark vs Express “pur”

Express :

- minimal, flexible
- devient vite “désorganisé” si le projet grossit (middlewares dispersés, logique métier pas structurée)

NestJS :

- structure imposée (bonne pratique)
- scalable par modules
- très lisible et maintenable
- plus “verbeux” au début

Choix final : NestJS car on a des notions structurées : auth, admin, workspaces, invites, tasks, notifications, jobs... Donc un framework modulaire apporte de la robustesse.

Authentification JWT (Access + Refresh)

- L’API est sécurisée par **Bearer Token** (access token).
- **Refresh token** stocké en base sous forme **hashée** (sécurité en cas de fuite).

Benchmark vs sessions/cookies

- Cookies/sessions : bien pour apps monolithiques, plus compliqué cross-domain et scalable.
- JWT : standard pour API découplée, mobile-friendly, facile à intégrer.

Choix JWT car architecture découplée + besoin d’une API réutilisable.

Guards : JwtAuthGuard + AdminGuard

- **JwtAuthGuard** protège les routes nécessitant authentification.
- **AdminGuard** bloque les routes admin si l’utilisateur n’a pas `global_role=ADMIN`.

Sécurité plus “propre” qu’un check manuel dans chaque controller.

Validation DTO (ValidationPipe + class-validator)

- évite les payloads incohérents
- nettoie les champs inattendus (whitelist)
- garantit une API robuste même si le front change

1.4 Base de données : PostgreSQL (Neon) + Prisma + Migrations

1.4.1 Pourquoi PostgreSQL (vs MongoDB, MySQL, etc.) ?

Le domaine TaskFlow contient des données **fortement relationnelles** :

- Workspaces → Members → Projects → Tasks → Comments → Mentions → Notifications
- Permissions (OWNER/MANAGER/MEMBER)
- Intégrité (tâches liées à projets, membres, assignations)
- Historique et audit fields

PostgreSQL est excellent pour :

- relations + jointures (JOIN)
- contraintes (foreign keys, unique constraints)
- transactions
- cohérence forte des données

Benchmark vs MongoDB (NoSQL)

MongoDB est pertinent si :

- données peu relationnelles
- schéma très variable
- besoin de flexibilité document

Mais ici :

- les relations sont centrales (workspace membership, tasks assignment, invites)
- on a besoin d'intégrité et de contraintes (ex : unique workspace_id + user_id)
- les requêtes multi-collections deviennent plus complexes

Choix final : PostgreSQL car il correspond naturellement à un modèle relationnel (et réduit la complexité côté code).

Benchmark vs MySQL

MySQL est aussi relationnel, mais PostgreSQL est souvent préféré pour :

- fonctionnalités plus avancées (types, contraintes, requêtes)
- robustesse transactionnelle
- écosystème “SaaS/Enterprise” très courant

MySQL aurait fonctionné, mais **PostgreSQL** est plus “standard entreprise” pour ce type d'app.

1.4.2 Pourquoi Neon (vs Supabase, Render, Railway...) ?

Neon = PostgreSQL managé optimisé pour dev/projets modernes.

Neon apporte :

- PostgreSQL “pur” (compatible Prisma sans friction)
- provisioning rapide
- scaling simple
- idéal pour environnements dev (DB cloud accessible partout)

Benchmark vs Supabase

Supabase est très bon mais c'est une plateforme plus large :

- Auth, Storage, Realtime, Edge...
- parfois on adopte “tout Supabase”

Ici, on a déjà :

- Auth dans NestJS
- logique métier et règles d'accès custom (guards)
- notifications/jobs custom

Supabase aurait ajouté des composants “en plus” sans nécessité (et risquait de “dupliquer” l’auth et la logique).

Benchmark vs Render/Railway (DB)

Render/Railway offrent aussi Postgres, mais :

- Neon est très orienté serverless/dev
- mise en place souvent plus rapide pour une DB dédiée
- bon équilibre simplicité/performance

Choix final : Neon car on voulait une DB Postgres simple, indépendante, plug-and-play avec Prisma.

1.4.3 Pourquoi Prisma + migrations (vs TypeORM, Sequelize, SQL brut) ?

Prisma est choisi pour :

- Typage TypeScript très fort (moins d’erreurs runtime)
- Schéma central (`schema.prisma`) facile à lire
- Migrations intégrées et reproductibles
- Productivité élevée (rapide en délai court)

Benchmark vs TypeORM

TypeORM est puissant mais :

- configuration parfois plus lourde
- migrations et types moins “guidés”
- expérience développeur parfois moins fluide

Benchmark vs SQL brut

SQL brut :

- perf maximale et contrôle total
- beaucoup plus long à écrire
- risque d’erreurs + maintenance plus difficile
- migrations à gérer manuellement

Choix final : Prisma + migrations car :

- le projet nécessite de livrer vite (délai)
- mais en gardant une structure propre et maintenable
- et des migrations reproductibles (déploiement propre)

1.5 Concepts de robustesse : Soft delete + Audit fields + Enums

Soft delete (active / deleted)

Au lieu de supprimer physiquement une ligne :

- `deleted=1` et `active=0`
- conservation de l’historique
- évite de casser des relations (FK)

Benchmark vs hard delete :

- Hard delete est simple, mais détruit l’historique et peut casser des références.
- Soft delete est plus adapté au contexte entreprise (audit + traçabilité).

Audit fields (creator_id, updatator_id, etc.)

- trace qui a créé / modifié / supprimé
- utile pour conformité et debug
- cohérent avec un usage entreprise

Enums métiers (TaskStatus, WorkspaceRole, InviteStatus)

- évite des valeurs incohérentes (ex : status libre en string)
- renforce la qualité des données
- rend le code plus sûr et lisible

Structure du Backend (Domain Driven Design léger)

2.1 Principe : DDD “léger” orienté domaines

Le backend NestJS est organisé par **domaines fonctionnels** (features), au lieu d’une séparation purement technique (ex : “controllers/ services/ dto” tous mélangés).

Chaque domaine encapsule :

- **Controller** : expose les routes REST (HTTP)
- **Service** : contient la logique métier (use-cases)
- **DTO** : définit et valide les payloads entrants (class-validator)
- **Module** : assemble les dépendances (providers/controllers/imports)

Objectif :

- lisibilité (on sait où chercher)
- évolutivité (ajouter un domaine ne casse pas le reste)
- maintien de règles métier cohérentes (permissions, audit, soft delete)

2.2 Arborescence type

Exemple de structure (par domaine) :

- auth/
- workspaces/
- projects/
- tasks/
- comments/
- notifications/
- attachments/
- admin-users/
- dashboard/

Chaque dossier contient généralement :

```
domain/  
  domain.controller.ts  
  domain.service.ts  
  dto/  
    *.dto.ts  
  domain.module.ts
```

2.3 Rôle de chaque couche

A) Controller (Interface HTTP)

Le **Controller** :

- définit les routes (@Get, @Post, @Patch, @Delete)
- applique les guards (@UseGuards(JwtAuthGuard, ...))
- récupère les paramètres (@Param, @Query, @Body)

- délègue au service (aucune logique métier lourde ici)

Bonnes pratiques appliquées :

- routes claires et REST
- accès protégé par guards
- validation automatique via DTO + ValidationPipe global

B) Service (Logique métier)

Le **Service** :

- centralise la logique métier (création, assignation, permissions, etc.)
- orchestre Prisma (transactions si nécessaire)
- applique les règles d'accès (ou appelle des helpers dédiés : `WorkspaceAccessService`, guards)

Avantages :

- logique réutilisable (future CLI / jobs / endpoints)
- testabilité facilitée
- évite la duplication dans les controllers

C) DTO (Validation & Contrats)

Les **DTO** :

- définissent le “contrat” d’entrée attendu
- valident les données via `class-validator`
- sécurisent l’API (pas de champs surprises si `whitelist: true`)

Exemple de règles typiques :

- email valide

- password min 6 caractères
- enums (roles/status) stricts

D) Module (Assemblage)

Le **Module** :

- déclare les controllers et providers du domaine
- importe les dépendances (ex : `PrismaModule`)
- rend le domaine indépendant et “plug-and-play”

Résultat : domaines isolés, intégration propre dans `AppModule`.

2.4 Exemple concret par domaine

auth/

Responsable de :

- login / refresh / logout
- endpoints /auth/me
- gestion JWT + stockage refresh token hashé
- éventuellement “first password” (reset_password)

Contenu typique :

- `auth.controller.ts`
- `auth.service.ts`
- `jwt.strategy.ts`, `jwt.guard.ts`
- `dto/`

workspaces/

Responsable de :

- création et gestion workspaces
- rôles workspace (OWNER/MANAGER/MEMBER)
- membres, invitations (invites), endpoints “me” (rôle dans workspace)

Contenu typique :

- `workspaces.controller.ts`
- `workspaces.service.ts`
- `workspace-invites.*`
- `dto/`

tasks/

Responsable de :

- CRUD tâches
- assignation / réassignation
- actions métier : ack, close, desist
- events/tasks history

notifications/

Responsable de :

- liste “mes notifications”
- marquer comme lue
- support des notifications système (deadline, invite workspace, etc.)

dashboard/

Responsable de :

- endpoint agrégé /dashboard
- regroupe des infos issues de plusieurs domaines (tasks, notifications, workspaces)
- évite au front de faire 5 appels séparés

2.5 Benchmark : pourquoi cette approche “DDD léger” ?

Vs structure “par type” (controllers/ services/ dto séparés globalement)

Alternative :

- controllers/, services/, dto/ au même niveau pour toute l’app

Limites :

- difficile de naviguer (tu cherches un use-case → tu sautes entre dossiers)
- risque de duplication
- augmente la confusion quand l’app grandit

Notre approche (par domaine) :

- regroupe tout ce qui concerne une feature au même endroit
- facilite la maintenance et l’évolution

Vs “DDD strict” (Entities, Aggregates, Repositories séparés)

DDD strict :

- très robuste pour gros systèmes
- trop lourd pour un MVP / délai court

- demande beaucoup de classes et abstractions

DDD léger :

- garde l'essentiel (domaine = feature)
- conserve un code simple et livrable rapidement
- laisse la porte ouverte à une complexification future si nécessaire

Sécurité

3.1 Authentification

JWT Access Token (15 min)

- Utilisé pour authentifier chaque requête API via l'en-tête :
Authorization: Bearer <accessToken>
- Durée courte (15 minutes) pour limiter l'impact en cas de fuite de token.
- Contient les informations minimales nécessaires :
 - sub (user_id)
 - email
 - name
 - (optionnel) global_role si on veut l'utiliser côté guards

Pourquoi 15 min ? (benchmark)

- **Plus court (ex: 5 min)** : meilleure sécurité, mais plus de refresh → plus de friction et charge serveur.
- **Plus long (ex: 1h)** : moins de refresh, mais un token volé reste exploitable plus longtemps.
- **Compromis choisi (15 min)** : standard en production pour équilibrer UX + sécurité.

JWT Refresh Token (7 jours)

- Sert uniquement à récupérer un nouveau access token quand celui-ci expire.
- Durée longue (7 jours) pour une UX fluide (pas de reconnexion fréquente).
- Stocké côté client (localStorage dans votre MVP).

Pourquoi 7 jours ? (benchmark)

- **24h** : plus secure, mais relogin fréquent.
- **30 jours** : meilleur confort, mais risque plus long si token fuit.
- **7 jours** : compromis réaliste + pratique en projet académique/MVP.

Refresh tokens hashés en base (SHA-256)

Au login :

1. Le refresh token est généré et renvoyé au client.
2. En base, on ne stocke **pas** le refresh token en clair, mais son hash :
 - a. `token_hash = sha256(refreshToken)`

Avantages :

- Si la base de données fuite, l'attaquant ne récupère pas les refresh tokens.
- Possibilité de révocation (logout, rotation, audit).

Benchmark vs stockage en clair

- **En clair** : plus simple, mais énorme risque si DB compromise.
- **Hashé (choix retenu)** : standard sécurité, simple à implémenter, très efficace.

3.2 Autorisation

Rôles globaux (plateforme)

- ADMIN : accès aux routes d'administration (ex: gestion utilisateurs)
- USER : utilisateur normal

Utilisation typique :

- AdminGuard contrôle `req.user.global_role === 'ADMIN'`
- protégé par JwtAuthGuard + AdminGuard

Benchmark vs permissions fines

- Permissions fines (RBAC/ABAC) : très robuste, mais coûteux à maintenir.
- **Rôles simples (choix MVP)** : suffisant, clair, rapide à auditer.

Rôles workspace (niveau domaine)

Un utilisateur peut avoir un rôle différent selon le workspace :

- OWNER : gestion complète (membres, projets, tâches, paramètres)
- MANAGER : gestion projets/tâches (assign/reassign, edit/delete TODO)
- MEMBER : exécution (accuser réception, clôturer, se désister)

Ces rôles sont stockés dans `WorkspaceMember`.

Benchmark : global role vs workspace role

- Tout en global role : simple mais pas réaliste (un user peut être admin partout par erreur).
- **Rôles par workspace (choix retenu)** : plus proche des outils réels (Notion, Asana, Jira).

Guards NestJS pour protéger les routes

Mécanisme standard NestJS :

- `JwtAuthGuard` : vérifie token + injecte `req.user`
- `AdminGuard` : bloque si pas ADMIN
- contrôles métier supplémentaires dans les services (ex: OWNER only)

Pourquoi guards + vérifs service ? (benchmark)

- Guards seuls : bien mais parfois insuffisant (permissions contextuelles).
- Services seuls : possible mais répétitif et moins centralisé.
- **Mix Guards + règles en services (choix retenu)** :
 - Guard = “porte d’entrée”
 - Service = règles métier contextualisées (`workspaceId`, `projet`, `task...`)

3.3 Premier login sécurisé (reset_password)

Champ reset_password

Ajout dans User :

- `reset_password = 0` → l'utilisateur **doit changer** son mot de passe
- `reset_password = 1` → accès normal

Scénario :

1. Admin crée un utilisateur avec un mot de passe temporaire.
2. À la première connexion :
 - a. si `reset_password = 0` → on laisse le login passer **mais** on force une redirection vers une page “Changer mot de passe”
3. Une fois changé :
 - a. on met `reset_password = 1`

Pourquoi laisser la connexion passer ?

- On a besoin d'un utilisateur authentifié (JWT) pour sécuriser l'endpoint "set password".
- Ça évite d'exposer un endpoint non-auth qui modifierait le mot de passe.

Endpoint dédié (ex: PATCH /auth/first-password)

- Protégé par JwtAuthGuard
- Vérifie que `reset_password === 0`
- Hash le nouveau password (bcrypt)
- Met `reset_password = 1`

Benchmark vs "lien reset email"

- Lien email type "forgot password" :
 - très sécurisé en prod
 - nécessite SMTP, tokens, pages dédiées
- **reset_password flag (choix MVP)** :
 - rapide, simple, efficace
 - répond à la contrainte "première connexion"
 - idéal pour projet avec délai court

Fonctionnalités principales

4.1 Gestion des Workspaces

Objectif : permettre d'organiser le travail par espaces indépendants.

Ce qui est fait

- Création d'un workspace
- Liste des workspaces accessibles par l'utilisateur connecté (/workspaces/me)
- Paramètres workspace :

- renommage
- activation/désactivation (**toggle-active**)
- soft delete

Choix techniques

- **WorkspaceMember** (table de liaison) pour gérer un rôle différent par workspace (OWNER/MANAGER/MEMBER)
- Contrôle d'accès : un utilisateur doit être membre pour accéder aux infos/membres/projets du workspace

4.2 Gestion des membres

Objectif : maîtriser qui a accès à un workspace.

Ce qui est fait

- Liste des membres d'un workspace (GET /workspaces/:workspaceId/members)
- Retrait d'un membre (DELETE /workspaces/:workspaceId/members/:memberId)
- Règles d'autorisation :
 - OWNER : tout gérer
 - MANAGER : gestion tâches/projets (selon règles)
 - MEMBER : exécution des tâches

Choix techniques

- Vérifs d'accès en service (métier) + Guard JWT (sécurité)
- Rôles stockés en DB = logique stable côté back (pas seulement côté UI)

4.3 Invitations

Objectif : inviter un utilisateur à rejoindre un workspace sans l'ajouter directement.

Ce qui est fait

- Création d'invitation (POST /workspaces/:workspaceId/invites)
- Liste des invitations (GET /workspaces/:workspaceId/invites)
- Révocation (DELETE /workspaces/:workspaceId/invites/:inviteId)

Choix techniques

- Token d'invitation généré aléatoirement, stocké **hashé (SHA-256)** en base
- Statuts d'invitation via enum `InviteStatus` : PENDING, ACCEPTED, REVOKED, EXPIRED
- Lien d'invitation généré côté backend (MVP), utilisable par le front

Pourquoi hash token ?

Même logique que refresh tokens : si DB compromise, le token n'est pas récupérable en clair.

4.4 Projets

Objectif : structurer le travail dans un workspace.

Ce qui est fait

- Création projet dans un workspace (POST /workspaces/:workspaceId/projects)
- Liste des projets d'un workspace (GET /workspaces/:workspaceId/projects)
- Détails + mise à jour + suppression soft (/projects/:projectId)

Choix techniques

- Un projet appartient à **un workspace**
- Un owner de projet est stocké (traçabilité : responsable principal)

4.5 Tâches

Objectif : gérer le cycle de vie complet des tâches.

Ce qui est fait

- CRUD tâches par projet (/projects/:projectId/tasks, /tasks/:taskId)
- Statuts TaskStatus :
 - TODO → DOING → DONE
 - - DESISTE (désistement)
- Assignment (PATCH /tasks/:taskId/assign)
- Accuser réception / démarrer (PATCH /tasks/:taskId/ack)
- Clôturer (PATCH /tasks/:taskId/close)
- Désistement (PATCH /tasks/:taskId/desist)
- Réassignation (PATCH /tasks/:taskId/reassign)

Règles d'accès (exemples)

- MEMBER :
 - peut accuser réception et clôturer **uniquement s'il est assigné**
 - peut se désister s'il est assigné
- MANAGER/OWNER :
 - peut assigner et réassigner
 - peut modifier/supprimer une tâche **tant qu'elle est TODO**

Choix techniques

- On conserve `assignee_id_old` lors d'un désistement (audit & traçabilité)
- Désistement crée un état exploitable par l'UI (statut DESISTE) + infos `desist_reason`, `desist_comment`

4.6 Commentaires avec mentions

Objectif : discussions contextualisées au niveau d'une tâche + ping ciblé.

Ce qui est fait

- Ajout commentaire (POST `/tasks/:taskId/comments`)
- Liste commentaires (GET `/tasks/:taskId/comments`)
- Mentions via table `CommentMention` :
 - relation commentaire ↔ utilisateur mentionné

Choix techniques

- Table dédiée `CommentMention` (plutôt qu'un champ texte) :
 - permet requêtes propres : "qui est mentionné ?"
 - permet notifications ciblées fiables

4.7 Notifications

Objectif : centraliser l'activité importante (deadline proche, invitations, mentions, etc.).

Ce qui est fait

- Liste des notifications de l'utilisateur connecté (GET `/notifications/me`)

- Marquer comme lue (PATCH /notifications/:notificationId/read)
- Compteur non-lu exploitable par le Header

Choix techniques

- read en 0/1 (compatible soft delete + simplicité)
- Index DB sur user_id + read pour accélérer “unread count” et listing

4.8 Dashboard global

Objectif : donner une vue rapide “priorités + alertes + accès rapide”.

Ce qui est fait

- Endpoint GET /dashboard qui renvoie :
 - compteurs (overdue, dueSoon, TODO, notifications non-lues)
 - aperçu tâches urgentes
 - workspaces récents

Choix techniques

- Un endpoint agrégateur “dashboard” évite au front d’appeler 6 endpoints séparés
- Permet un affichage plus rapide et cohérent (un seul aller-retour réseau)

4.9 Upload pièces jointes

Objectif : associer des fichiers à une tâche.

Ce qui est fait

- Upload fichier pour une tâche (POST /tasks/:taskId/attachments)

- Liste des pièces jointes d’une tâche (GET /tasks/:taskId/attachments)
- Suppression (DELETE /attachments/:attachmentId)
- Fichiers servis statiquement via /uploads/...

Choix techniques

- Stockage local en MVP (simple, rapide, pas de dépendance S3)
- Méta-données enregistrées en DB : file_name, url, content_type, file_size
- Possibilité future de migrer vers S3 sans changer le modèle fonctionnel

4.10 Cron job : notification “deadline proche”

Objectif : prévenir automatiquement un utilisateur si une tâche arrive à échéance.

Ce qui est fait

- Job planifié @nestjs/schedule toutes les heures
- Sélection des tâches dont la deadline est dans les prochaines 24h
- Création d’une notification si elle n’existe pas déjà

Anti-duplicate

- type unique basé sur :
TASK_DEADLINE_SOON:{taskId}:{YYYY-MM-DD}
- Vérifie existence avant création

Choix techniques

- Prisma findMany + boucle :
 - lisible, maintenable, suffisant MVP
- UTC pour la clé jour :
 - évite incohérences selon timezone serveur

Choix techniques importants

Pourquoi NestJS ?

NestJS a été choisi comme framework backend car il fournit une base “entreprise-ready” dès le départ, sans réinventer l’architecture.

Points clés

- **Architecture propre et modulaire**
 - Un module par domaine (auth, workspaces, tasks, etc.)
 - Séparation claire Controller / Service / DTO
 - Facilite la maintenance et l’évolution (ajout de features sans casser le reste)
- **Support natif de l’écosystème sécurité**
 - Intégration robuste de **Passport**, **JWT**, Guards, Decorators
 - Contrôle d’accès clair via **JwtAuthGuard**, **AdminGuard**, etc.
- **Standardisation**
 - Les patterns NestJS sont proches des standards backend (Spring-like)
 - Onboarding plus simple pour d’autres devs
- **Scalabilité organisationnelle**
 - Même si le projet reste “MVP”, la structure est déjà prête pour grandir (services, modules, tests, etc.)

Benchmark rapide

- vs Express “pur” : plus flexible mais architecture à construire soi-même (risque de dette technique + incohérences).
- vs Fastify : très performant mais moins “opinionated” sur l’architecture globale.
- vs Django/Flask (Python) : bon mais on perd l’écosystème TypeScript partagé avec le front.

Pourquoi Prisma ?

Prisma est utilisé comme ORM pour gagner en productivité tout en gardant une forte sécurité de typage.

Points clés

- **Typage strict TypeScript**
 - Les modèles Prisma génèrent des types auto → moins d’erreurs runtime
 - Autocomplétion très efficace
- **Migrations simples et traçables**
 - Les changements de schéma sont versionnés
 - Historique clair des évolutions DB (important en équipe)
- **Productivité élevée**
 - CRUD rapide
 - Relations faciles (workspace → members → user, etc.)
- **Sécurité**
 - Moins de SQL raw = moins de risques d’injection
 - Contrôles cohérents via schéma

Benchmark rapide

- vs TypeORM : plus “class-based”, mais migrations souvent plus fragiles et typage moins fiable en pratique.
- vs Sequelize : mature mais typage TS moins strict / DX moins moderne.
- vs SQL brut : performant mais coûteux en temps + maintenance, surtout sur un MVP.

Pourquoi Soft Delete ?

Le **soft delete** (champs `active` / `deleted`) a été choisi pour préserver l'historique et faciliter l'audit.

Points clés

- **Historique conservé**
 - On ne perd pas les données (utile pour retrouver un workspace/projet/tâche supprimé)
- **Pas de suppression destructrice**
 - Évite des erreurs irréversibles en production
- **Audit et traçabilité**
 - Avec `deleted_date`, `deletor_id`, on sait qui a fait quoi et quand
- **Protection des relations**
 - Une suppression “hard” casse facilement des FK ou rend les historiques incohérents

Benchmark rapide

- vs hard delete : plus “propre” en DB mais risque de perte de données et problèmes relationnels.
- vs archive séparée : plus complexe (tables miroir, jobs de transfert), pas nécessaire pour un MVP.

Pourquoi PostgreSQL plutôt que MongoDB ?

PostgreSQL est plus adapté ici car le projet est très relationnel.

Points clés

- **Données fortement relationnelles**
 - Workspaces ↔ Members ↔ Users ↔ Projects ↔ Tasks ↔ Comments ↔ Notifications

- Les jointures et contraintes d'intégrité sont centrales
- **Transactions**
 - Création workspace + ajout membre OWNER dans une transaction (cohérence garantie)
- **Contraintes & intégrité**
 - @@unique([workspace_id, user_id]), enums, relations → sécurité forte côté DB
- **Requêtes analytiques**
 - Dashboard, counts, overdue/dueSoon → très naturel en SQL

Benchmark rapide

- MongoDB : très bon pour documents flexibles, mais ici les relations + contraintes + reporting rendent SQL plus fiable et plus simple.
- MySQL : possible aussi, mais Postgres est souvent préféré pour ses fonctionnalités avancées et son écosystème.

Pourquoi Neon (PostgreSQL cloud) plutôt que Supabase / Render ?

Neon a été choisi pour un déploiement PostgreSQL cloud simple, rapide et adapté à un MVP.

Points clés

- **PostgreSQL managé très simple**
 - Création DB rapide, connexion directe via DATABASE_URL
- **Très adapté dev + production légère**
 - Suffisant pour un MVP (petite charge)
- **Coût/gestion**
 - Pas besoin de gérer un serveur DB soi-même

Benchmark rapide

- Supabase :
 - Excellent produit, mais plus “plateforme” (auth, storage, realtime) → surdimensionné si on veut maîtriser notre auth/JWT et notre architecture.
- Render :
 - Très bien pour héberger des services, mais DB Postgres peut coûter plus cher selon la config, et la gestion peut être un peu moins “DB-first” que Neon.
- Railway :
 - Simple aussi, mais modèle de pricing et stabilité peuvent varier selon usage.

Instructions de Déploiement

Prérequis

- **Node.js ≥ 18** (recommandé : 20+)
- **PostgreSQL (local) ou Neon**
- **npm (ou pnpm)**

Installation

1 Cloner le projet

```
git clone <repo-url>
cd taskflow
```

2 Installer les dépendances

Backend

```
cd apps/api  
npm install
```

Frontend

```
cd ../web  
npm install
```

Configuration Backend (NestJS)

1 Créer un fichier .env dans apps/api

```
DATABASE_URL="postgresql://USER:PASSWORD@HOST:5432/DBNAME?sslmode=require"
```

```
JWT_ACCESS_SECRET=supersecret1  
JWT_REFRESH_SECRET=supersecret2
```

```
API_PORT=3001  
FRONT_URL=http://localhost:3000
```

Notes

- Si tu utilises **Neon**, tu colles directement le DATABASE_URL fourni.
- `sslmode=require` est souvent nécessaire en cloud (Neon).

Migration Base de données (Prisma)

Depuis apps/api :

```
npx prisma generate  
npx prisma migrate dev
```

`generate` : génère le client Prisma (types TypeScript).
`migrate dev` : applique les migrations + met à jour la DB.

Lancer le backend

Toujours dans `apps/api` :

```
npm run dev
```

API disponible sur :
<http://localhost:3001>

Configuration Frontend (Next.js)

Créer un fichier `.env.local` dans `apps/web`

```
NEXT_PUBLIC_API_URL=http://localhost:3001
```

Lancer le frontend

Depuis `apps/web` :

```
npm run dev
```


Application disponible sur :
<http://localhost:3000>

Utilisation

1) Créer un premier utilisateur (bootstrap)

Endpoint

POST /auth/register

Objectif

Créer le **premier compte** (souvent un compte ADMIN/OWNER initial selon ta stratégie).

Ensuite, tu peux créer d'autres utilisateurs via l'admin ou invitations.

Exemple (JSON)

```
{  
  "email": "admin@taskflow.com",  
  "password": "Admin123!",  
  "name": "Admin"  
}
```

2) Connexion

Endpoint

POST /auth/login

Objectif

Authentifier l'utilisateur et récupérer les tokens :

- accessToken (court, utilisé pour les appels API)
- refreshToken (long, utilisé pour renouveler l'accès)

Exemple (JSON)

```
{  
  "email": "admin@taskflow.com",  
  "password": "Admin123!"  
}
```

Réponse typique

```
{  
  "accessToken": "xxx",  
  "refreshToken": "yyy"  
}
```

Utilisation dans les requêtes suivantes :

Ajouter le header :

Authorization: Bearer <accessToken>

3) Créer un workspace

Endpoint

POST /workspaces

Conditions

- Utilisateur connecté (JWT requis)

Exemple (JSON)

```
{  
  "name": "Workspace Marketing"  
}
```

Résultat

- Le workspace est créé
- Le créateur est automatiquement ajouté comme **OWNER**

4) Inviter des membres dans un workspace

Endpoint

POST /workspaces/:workspaceId/invites

Conditions

- JWT requis
- Rôle nécessaire : **OWNER** (ou OWNER/MANAGER selon ta règle)
- Une invitation est créée et peut aussi générer une notification côté utilisateur

Exemple (JSON)

```
{  
  "email": "user@taskflow.com",  
  "role": "MEMBER"
```

```
}
```

Résultat typique

- Invitation créée (status PENDING)
- (Optionnel) un lien d'invitation peut être renvoyé côté front (ex: invite_link)

5) Créer des projets et des tâches

Créer un projet

Endpoint

POST /workspaces/:workspaceId/projects

Exemple (JSON)

```
{  
  "name": "Refonte Site Web",  
  "description": "Projet de refonte UI/UX"  
}
```

Créer une tâche

Endpoint

POST /projects/:projectId/tasks

Exemple (JSON)

```
{
  "title": "Créer la maquette Figma",
  "description": "Maquette page d'accueil",
  "priority": "HIGH",
  "deadline": "2026-02-20T18:00:00.000Z",
  "assignee_id": "cuid_user_id_optionnel"
}
```

8 Points d'Amélioration possibles

1) Tests (qualité & fiabilité)

- **Tests unitaires** (services, helpers, règles métier)
- **Tests d'intégration** (API + DB via Prisma)
- **Tests E2E** (parcours complet : login → workspace → tâche → notification)

Pourquoi ? Réduit les régressions et sécurise les évolutions.

2) Documentation API (Swagger / OpenAPI)

- Ajouter @nestjs/swagger pour générer automatiquement :
 - endpoints
 - DTO
 - codes d'erreurs
 - auth Bearer

Pourquoi ? Facilite l'intégration, le debug, et la maintenance.

3) Dockerisation complète

- Dockerfile pour apps/api + apps/web
- docker-compose.yml incluant :
 - API
 - Front
 - PostgreSQL (si non utilisé via Neon)
 - (optionnel) Adminer/pgAdmin

Pourquoi ? Déploiement reproductible et simple pour n'importe quel environnement.

4) CI/CD (automatisation)

- GitHub Actions / GitLab CI :
 - lint + build
 - tests
 - migrations Prisma
 - déploiement automatique (Neon + Vercel/Render/Fly.io)

Pourquoi ? Standard “entreprise”, évite les déploiements manuels à risque.

5) Rate limiting & sécurité API

- @nestjs/throttler (limitation par IP / route)

- Protection brute-force sur /auth/login
- Headers sécurité (helmet)
- Validation stricte des payloads (déjà en place via ValidationPipe)

Pourquoi ? Empêche les abus et renforce la sécurité.

6) Logs centralisés & observabilité

- Logger structuré (pino/winston)
- Centralisation (ex: Logtail, Datadog, Grafana Loki)
- Ajout de métriques (latence, erreurs, jobs cron)
- Alerting (erreurs 500, DB down, cron failing)

Conclusion

TaskFlow est une application SaaS modulaire conçue selon des standards d'architecture modernes et orientés entreprise.

Elle repose sur :

- **Une architecture REST claire et découplée**
Frontend (Next.js) séparé du Backend (NestJS), facilitant la scalabilité et la maintenance.
- **Une sécurité robuste basée sur JWT**
 - Access Token court (15 min)
 - Refresh Token sécurisé et hashé en base
 - Guards NestJS pour protéger les routes sensibles
- **Un système de rôles hiérarchiques structuré**
 - Rôles globaux : ADMIN / USER
 - Rôles Workspace : OWNER / MANAGER / MEMBERPermettant un contrôle fin des permissions.

- **Audit et traçabilité complète**
 - Soft delete (active / deleted)
 - Champs d'audit (creator_id, updator_id, deleted_date, etc.)
 - Historique des actions via TaskEvent
- **Notifications automatiques et intelligentes**
 - Mentions utilisateurs
 - Invitations workspace
 - Cron job pour deadlines proches
 - Suivi d'activité en temps réel
- **Conception orientée entreprise**
 - Modularité forte (DDD léger)
 - Typage strict via Prisma + TypeScript
 - Structure prête pour CI/CD et Docker
 - Séparation claire des responsabilités

Vision globale

TaskFlow n'est pas seulement une application de gestion de tâches :

c'est une **base SaaS évolutive**, pensée pour :

- La sécurité
- La scalabilité
- La maintenabilité
- La traçabilité métier

Elle peut facilement évoluer vers :

- Multi-tenancy avancé
- Système de facturation
- Permissions fines par projet
- Microservices à grande échelle

